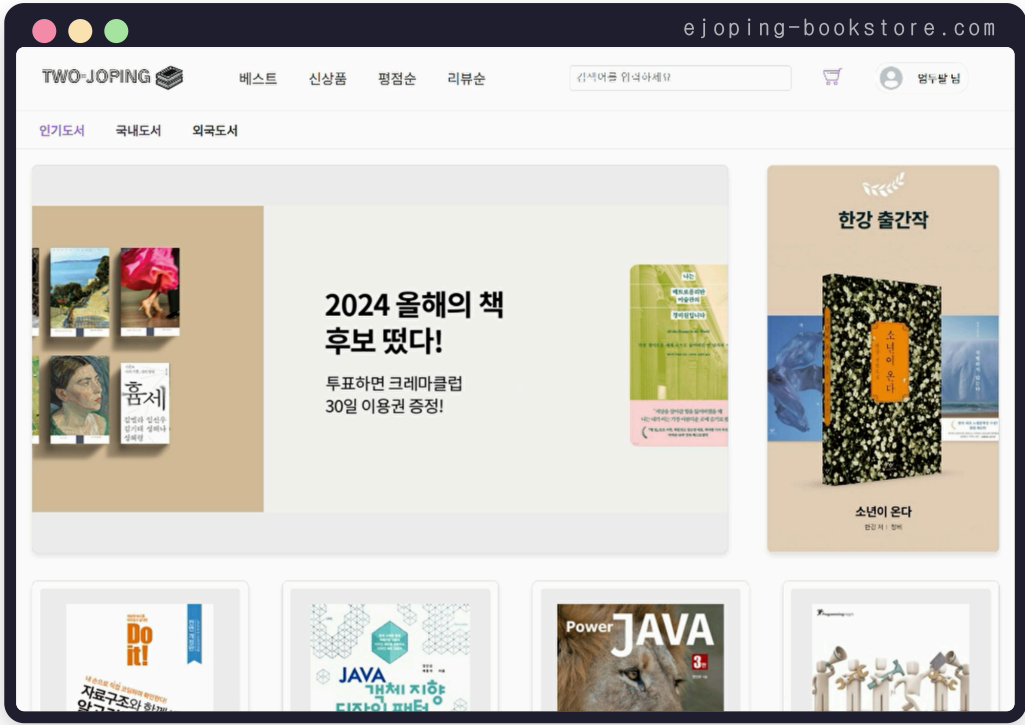
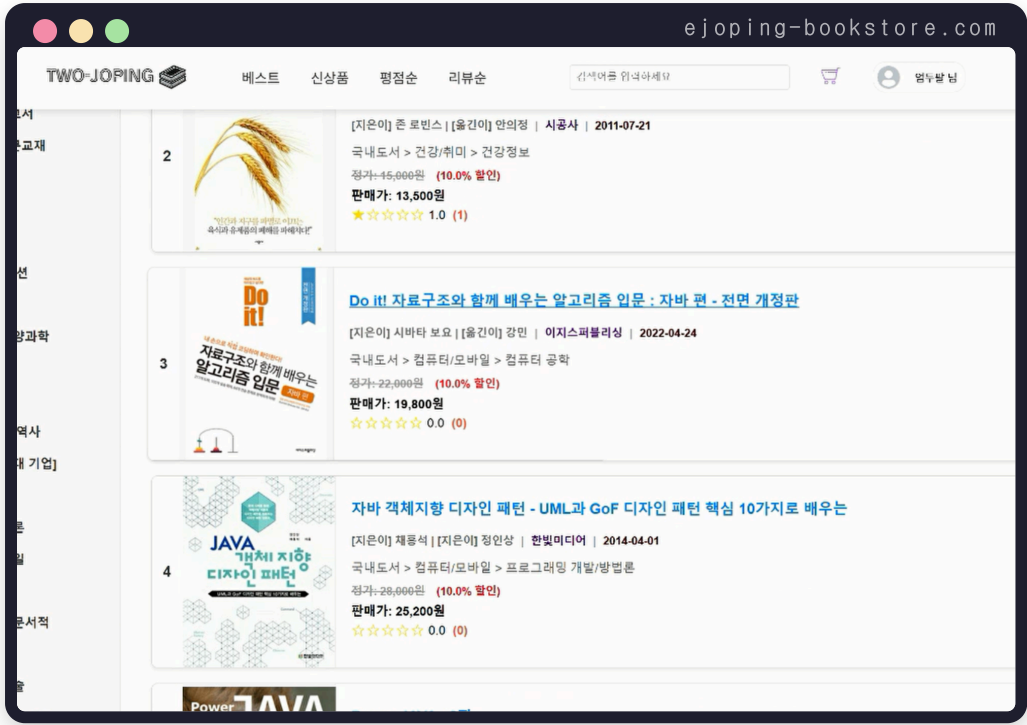




서비스

온라인 도서 쇼핑몰 (대용량 커머스 플랫폼)

Spring Boot + Thymeleaf 기반 도서 커머스 / Elasticsearch + Nori 형태소 분석기로 검색 엔진 도입

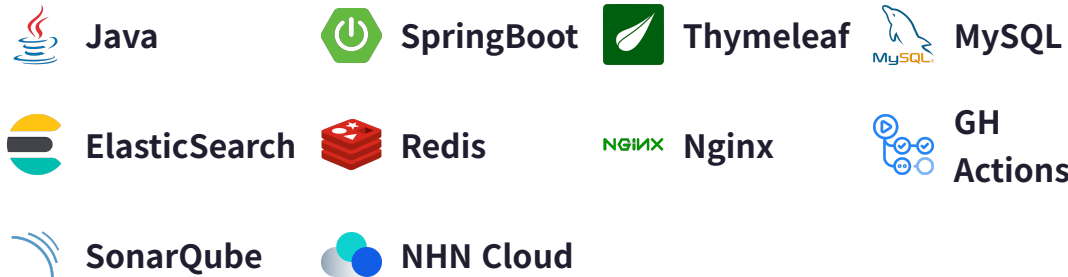
GitHub Repository

작업 기간

2024. 10. 14 - 2024. 12. 13 (9주)

NHN Academy 7기 온라인 도서 쇼핑몰 프로젝트

기술 스택



구성원

총 8명

FullStack 6명 (본인 포함) · Infra 2명

역할 — FullStack

Backend

검색 엔진 도입

MySQL → Elasticsearch 마이그레이션 / nori + ngram + jaso 분석기 설계

도메인 API 설계·구현

ERD 설계 + Spring Boot REST API (카테고리·검색·기여자·배송)

Frontend

Thymeleaf layout

화면 구성 + 데이터 출력 + 담당 도메인 페이지 퍼블리싱

협업 표준화

대규모 팀(8인) 일정 관리

WBS 기반 분담 + API 기술 문서화 + 스프린트 회고 주도

프로젝트 회고

- **RDBMS 검색 한계 분석** — MySQL FULLTEXT 평균 1.2초 응답 / Slow Query 빈발
 - JOIN N+1 + ORDER BY 조합으로 LIKE 검색의 병목 체감
- **ElasticSearch 도입으로 구조 개선** — 검색 전용 인덱스 분리 구축
 - 한글 분석기 nori + n-gram + jaso 조합 설계
 - 복합 검색어 환경에서도 응답 약 **280ms (76% 단축)**
- **형태소 분석으로 검색 본질 학습**
 - "검색은 LIKE 가 아니다" — 인덱스 매핑·재색인·동의어 처리 경험
- **대규모 팀(8인) 협업 리드**
 - WBS · 코드 리뷰 · 스프린트 회고 주도로 생산성 관리

PROBLEM → SOLUTION

개발 단계 검색 응답 평균 **1,200ms** · Slow Query **30%** 발생. **LIKE '%키워드%'** 앞 와일드카드는 B-Tree 인덱스 사용 불가 → MySQL FULLTEXT 한계 확인 후 **Elasticsearch + 한글 형태소 분석기** 도입 + **Logstash JDBC** 로 10초 단위 자동 동기화.

Before vs After

BEFORE

QueryDSL .contains() + 4 LEFT JOIN

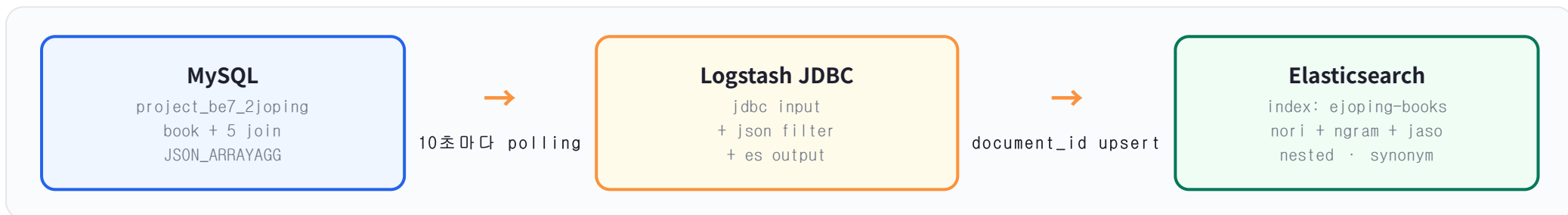
- ✗ Full Table Scan — LIKE '%keyword%' 앞 와일드카드로 인덱스 미사용
- ✗ 4 테이블 LEFT JOIN — contributor·tag·category 결과 행 폭발
- ✗ N+1 쿼리 — 도서별 기여자/카테고리 추가 조회
- ✗ 한국어 형태소 미지원 — '자바' 검색 시 '자바스크립트' 누락
- ✗ 관련성 정렬 미지원 — 단순 views 컬럼 정렬만 가능

AFTER

Elasticsearch + nori 형태소 분석기

- ✓ 역색인 + 형태소 토큰화 — Inverted Index 기반 빠른 검색
- ✓ multi_match + nested — 본문/카테고리/태그/기여자 동시 검색
- ✓ 관련성 점수(BM25) + 가중치 — title^10 / desc^3 / publisher^2
- ✓ 한글 분석기 nori + ngram + jaso — 부분 검색 · 자모 검색 지원
- ✓ Logstash 자동 동기화 — MySQL → ES 10초 polling upsert

인덱스 운영 — MySQL → Logstash → Elasticsearch



검색 쿼리 — multi_match + nested 결합

```
bookset/elastic/repository/impl/ElasticRepositoryImpl.java multi_match · nested · BM25

public Page<ElasticSearchResponseDto> searchByKeyword(String keyword, Long categoryId, Pageable pageable, String sort) {
    var req = new SearchRequest.Builder()
        .index(elasticConfig.getBookIndexName()) // "ejoping-books"
        .from((int) pageable.getOffset())
        .size(pageable.getPageSize())
        .query(q -> q.bool(b -> {

            // ① 본문 필드 multi_match - 가중치 + 4종 분석기 활용
            b.should(s -> s.multiMatch(m -> m.query(keyword).fields(
                "book_title^10", "book_title.ngram^3",
                "book_title.jaso", "book_title.synonym",
                "book_desc^3", "book_desc.ngram", "book_desc.synonym",
                "book_publisher^2", "book_publisher.ngram", "book_publisher.synonym"
            )));

            // ② 카테고리 nested - 트리 구조 책 분류 검색
            b.should(s -> s.nested(n -> n.path("book_category").query(
                cq -> cq.multiMatch(m -> m.query(keyword).fields(
                    "book_category.name^3", "book_category.name.ngram", "book_category.name.synonym"
                ))));

            // ③ 태그 nested · ④ 기여자 nested (저자/번역/출판) 동일 패턴
            // b.should(...book_tag...); b.should(...book_contributor...);

            return b;
        }));

    // 정렬 - popularity / newest / lowest_price / highest_price / rating / reviews
    sortCriteria.forEach(e -> req.sort(s -> s.field(f -> f.field(e.getKey()).order(...))));

    // → 평균 280ms · Slow Query 0건
    return elasticsearchClient.search(req.build(), Map.class);
}
```

76% ↓

평균 응답 시간

1,200ms → 280ms

0건

Slow Query

이전 30% → 0건

10초

Logstash 동기화

MySQL → ES 자동 upsert

1 쿼리

검색 호출

4 JOIN → multi_match + nested

한글 검색 3 분석기 조합

ejoping-books index

①

my_nori_analyzer

한국어 형태소 분석

```
tokenizer: nori_tokenizer
decompound_mode: "mixed" filter: ·
nori_part_of_speech (불용 품사 제거
E·J·MM...) · synonym_filter (자바=java)
· lowercase · trim · stop char_filter:
html_strip
```

"객체지향" → [객체, 지향, 객체지향] 형태소 분리. 불용 품사·동의어·HTML 태그 까지 정제하여 본문 검색 정확도 향상.

②

ngram_analyzer

부분 일치 / 자동완성

```
tokenizer: ngram_tokenizer min_gram: 2
max_gram: 3 filter: · 동일 (lowercase,
stop, nori_part_of_speech, synonym_filter)
```

"자바스크립트" → [자바, 바스, 스크, 크림, 립트, ...]
2~3자 토큰 생성. 중간 일부만 맞아도 인덱스 매칭.

③

suggest_(index/search)

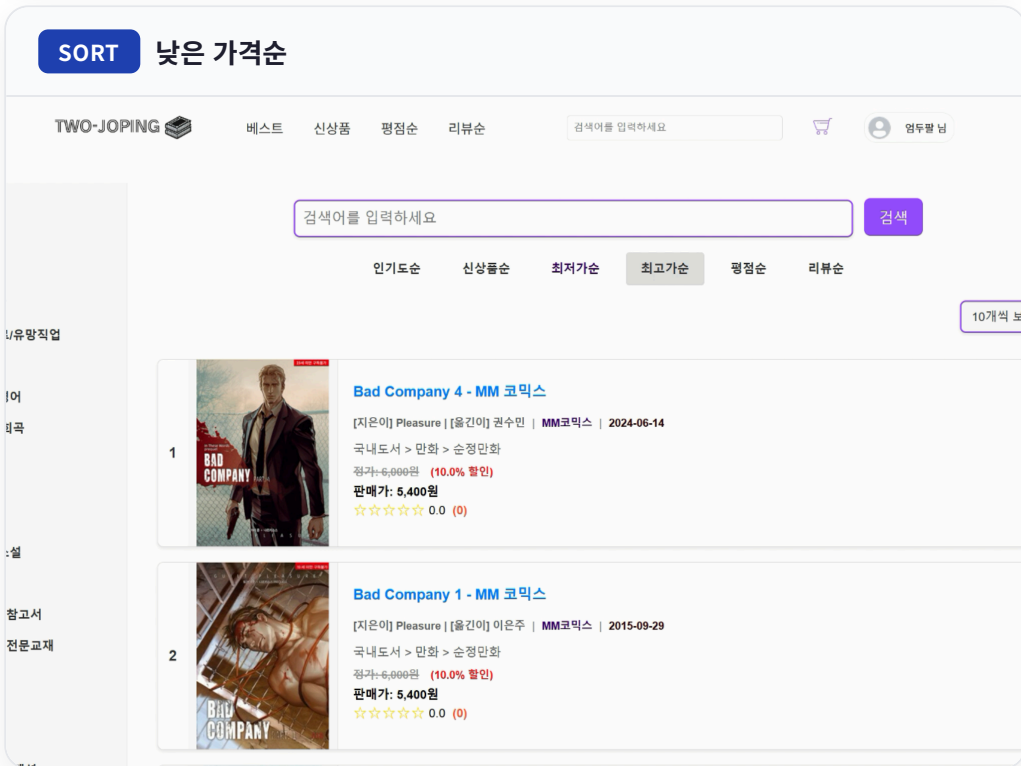
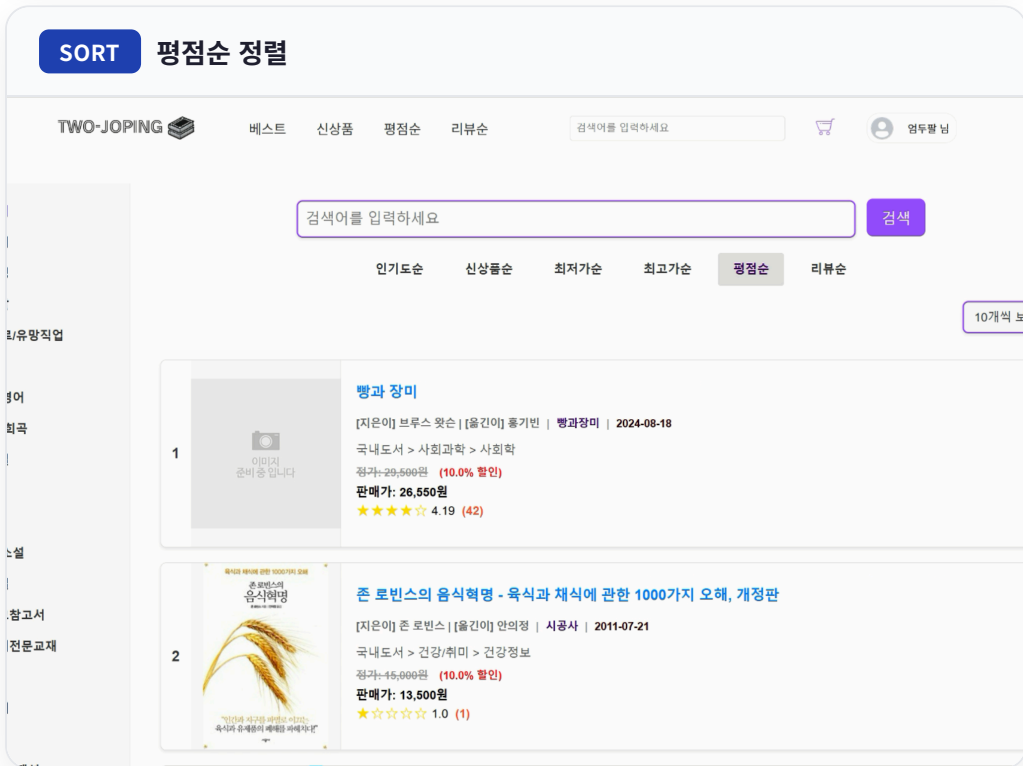
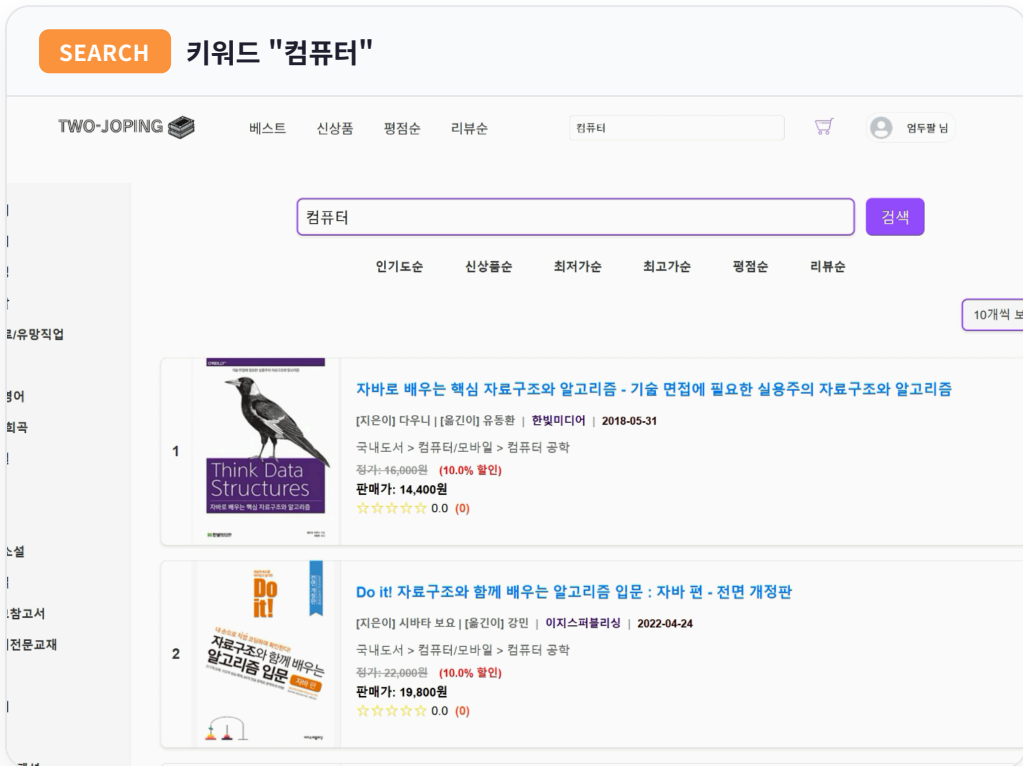
자모 검색 / 오타 허용

```
tokenizer: jaso_tokenizer mistype: true
· 오타 허용 chosung: index true / search
false filter (인덱스만): · edge_ngram
(1-50) suggest_filter
```

한글을 자모 분해. 색인엔 초성 포함으로 "자바스크립트" 같은 초성 검색 + 오타 보정 자동완성 지원.

검색 결과 화면 — 분석기 + 가중치 + 정렬 6 종

ejoping-bookstore.com



A

검색 인덱스 운영

- ▶ 운영 DB와 검색 전용 인덱스 분리 — 부하 격리
- ▶ Logstash JDBC input — MySQL 5 join + JSON_ARRAYAGG → ES upsert
- ▶ 정렬 6종: popularity · newest · price · rating · reviews
- ▶ BM25 + 가중치 — title^10 · desc^3 · publisher^2

B

8인 팀 협업 표준화

- ▶ WBS 기반 일정 관리 + 분담 및 마일스톤 추적
- ▶ API 기술 문서화 표준화 (Swagger + 명세서)
- ▶ 스프린트 회고 주도 — 매 스프린트 KPT 작성
- ▶ Pull Request 코드 리뷰 정책 + SonarQube 정적 분석